

EpochCut: Verifiable Attack-Path Closure for Dynamic Multi-Agent LLM Systems

Chandrasen Pandey, Vaibhav Tiwari, and Ashish Pandey

Abstract—Dynamic LLM agent systems can add agents, tools, memories, and links during execution, allowing a previously placed defense to become incomplete. We present EPOCHCUT, a control-plane mechanism that admits each topology change as a graph transaction. For every declared risk, it computes and independently verifies a minimum-cost vertex cut, binds the result to the prospective graph, and atomically activates topology and monitor routing. We prove conditional path closure under complete graph reporting and trusted enforcement. In a frozen evaluation of 12,000 online mutations, EPOCHCUT left 0/6,000 adversarial bypasses unmediated, reduced configured monitoring cost by 88.7% relative to full mediation, and achieved 1.62 ms p95 admission latency. A second evaluation connected three local tool-calling LLMs to a persistent sandboxed MCP service. Native agents committed 18/45 injected effects; EPOCHCUT blocked all 18 attempts, committed 0/45, and preserved exact benign completion at 16/18. A separately frozen hosted-model extension blocked all 3 observed injected attempts, committed 0/15, and retained 6/6 benign completion. The results establish attack resistance for the tested MCP paths, not universal production security.

Index Terms—agentic AI, multi-agent systems, attack graphs, complete mediation, runtime security, graph cuts, prompt injection

1 INTRODUCTION

Tool-using large language model (LLM) agents no longer operate as isolated chatbots. An orchestrator can spawn specialists, connect to new tool servers, write shared memory, delegate authority, and route intermediate results to other agents. This flexibility improves task coverage, but it also makes the security boundary a moving target. Prompt injection benchmarks such as AgentDojo [1], InjecAgent [2], and ASB [3] show that untrusted content can influence consequential tool calls. In a multi-agent system, the same influence can cross several principals before reaching a durable effect.

Existing defenses contribute important local mechanisms. CaMeL and Fides separate trusted control from untrusted data and enforce information-flow policies [4], [5].

AgentGuard applies attribute-based policies at tool boundaries [6]. SafeFlow propagates semantic risk across collaboration graphs [7], and the Agentic Principal Chain bounds delegated scope and composed actions [8]. Commit-time authorization protects the last boundary before a durable effect [9]. These mechanisms answer what a monitor should decide. They do not, by themselves, answer a different control-plane question: after the runtime changes its membership or topology, does every declared attack path still cross at least one such monitor?

This question is not hypothetical. Architecture alone can change multi-agent attack success by several fold [10]; colluding agents can make nominally independent agreement unreliable [11]; and the latest execution-centred systematization identifies dynamic membership and end-to-end path closure as open evaluation problems [12]. The classic complete-mediation principle requires every access to be checked, not merely the accesses known when a process started [13].

We present EPOCHCUT, a small trusted graph-epoch manager that preserves a declared form of complete mediation as an agent system changes. Figure 1 shows its two planes. The data plane contains agents, shared state, and protected effects. The control plane receives a proposed topology mutation, constructs the prospective influence graph, solves and verifies one weighted vertex cut per risk, emits hash-bound certificates, and only then commits the graph and monitor routing together.

This paper makes five contributions.

- We define *epoch-relative attack-path closure* for typed, dynamically changing multi-agent influence graphs. The property is explicit about sources, effects, interfaces, monitoring cost, and assumptions.
- We design a transactional admission protocol that computes per-risk minimum cuts, independently checks path separation, and binds monitor placement to the exact prospective graph and epoch.
- We prove that no declared source-to-effect path remains unmediated in an admitted epoch when the graph is complete, admission is serialized, and the selected monitors are trusted and correctly enforce their local policy.
- We release a deterministic reference implementation, tests, a frozen 60,000-row graph evaluation, raw outputs, analysis code, and publication figures. The

- C. Pandey is with the School of Computer Science, University of Petroleum and Energy Studies (UPES), Dehradun, Uttarakhand, India. E-mail: chandrasen.pandey@ddn.upes.ac.in.
- V. Tiwari is with the School of Computer Science, Bennett University, Greater Noida, Uttar Pradesh, India. E-mail: vaibhav.tiwari@bennett.edu.in.
- A. Pandey is with the Department of Data Science and Engineering, Manipal University Jaipur, Jaipur, Rajasthan, India. E-mail: ashish.pandey@jaipur.manipal.edu.

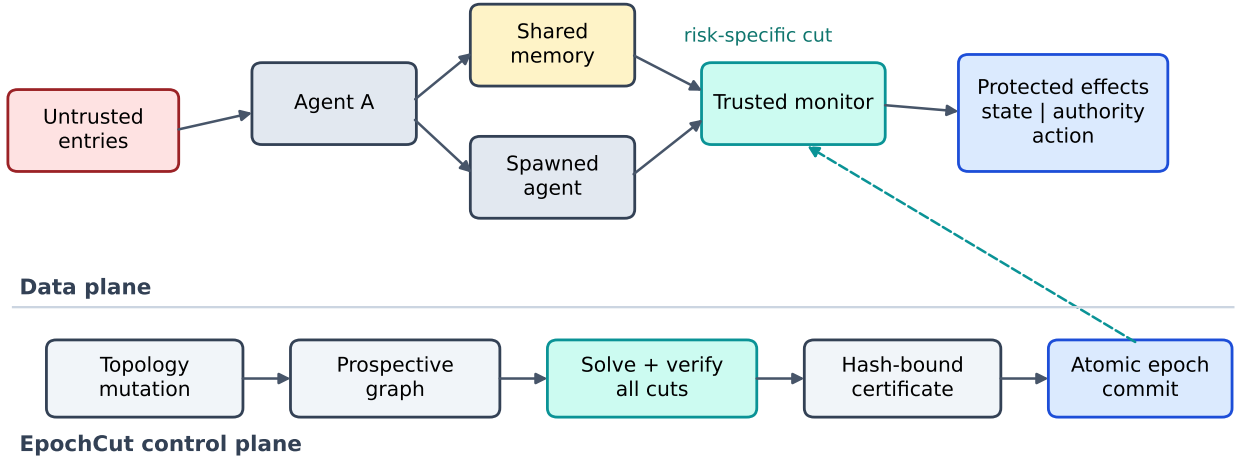


Fig. 1. EPOCHCUT separates dynamic agent execution from topology admission. A mutation becomes visible only after every declared source-to-effect path in the prospective graph intersects a verified, risk-specific monitor set.

evaluation separates path coverage from the quality of any natural language detector.

- We implement a real MCP studio adapter and evaluate 126 local-model and 42 hosted-model agent runs. The guarded deployments prevent all observed injected effects while preserving their native benign completion rates.

The scope is deliberately narrow. EPOCHCUT does not recognise malicious text, infer undeclared interfaces, or make a compromised monitor trustworthy. It supplies and verifies the placement condition under which an existing policy, capability check, taint gate, or human approval mechanism gets an opportunity to act.

The remainder of this paper is organized as follows. Section II reviews agent attack surfaces and enforcement mechanisms. Section III defines the system and threat model. Sections IV–VI present the design, security analysis, and implementation. Section VII evaluates graph reconfiguration and real MCP execution, Section VIII discusses deployment boundaries, and Section IX concludes the paper.

2 BACKGROUND AND RELATED WORK

2.1 Agent attack surfaces and benchmarks

Indirect prompt injection turns data returned by a tool, website, or message into instructions that steer the agent. AgentDojo contains realistic tasks and hundreds of task-attack pairs [1]; InjecAgent targets direct harm and data theft through tool-integrated agents [2]; and ASB spans prompt, tool, memory, and backdoor attacks [3]. Resource-amplification work further shows that a correct final answer can hide a costly malicious trajectory [14]. These studies motivate checking the complete execution rather than the final output alone.

Recent papers in *IEEE Transactions on Dependable and Secure Computing*, the target journal, reinforce the need for system-context evaluation. RedAgent adapts jailbreaks

to application prompts, tools, and tasks, whereas Poly-Jailbreak uses multi-agent optimization to expose closed-box multimodal safety failures [15], [16]. PILLM couples LLM generation to execution feedback for program testing, while FLUTE protects LLM inference inputs and parameters through secure two-party computation [17], [18]. These four same-journal studies establish closely related attack-generation, testing, and confidential-inference contexts. EPOCHCUT addresses the complementary runtime question of whether mediation remains on every declared attack path after the agent topology changes.

Multi-agent systems add communication, shared state, aggregation, and delegation. Empirical work finds that role allocation, topology, and memory visibility materially change the security outcome [10]. Secret-collusion results show why agreement cannot be treated as independent evidence without additional assumptions [11]. The recent A-I-R view traces adversarial influence through interaction interfaces to system-level risks and observes that most evaluated systems retain fixed membership [12]. Our benchmark varies the topology online and asks whether mediation remains complete after every change.

2.2 Local enforcement and workflow controls

CaMeL extracts trusted control flow before untrusted data enters the planner [4]; Fides formalises the class of policies enforceable by dynamic taint tracking [5]; and SafeFlow carries semantic taints across agents [7]. Authorization systems bound attributes, scope, budgets, or action composition [6], [8]. CommitGuard revalidates the authority witness at the durable-effect boundary [9], while Cordon places a transaction around a complete tool workflow [19]. These systems are complementary enforcement functions. EPOCHCUT reasons about whether their deployment points intersect all declared paths after a graph change.

Table 1 states this distinction without treating adjacent systems as interchangeable baselines. A language-level defense can be strong while its runtime attachment is incom-

plete; a perfectly placed monitor can also make a poor decision. End-to-end security needs both properties.

2.3 Graph separation

A vertex cut intersects every directed path between a source set and a sink set. Node splitting reduces a weighted vertex cut to a standard maximum-flow problem. Our prototype uses a deterministic Edmonds-Karp solver [20]. Graph separation is classical; the contribution is the security abstraction and transactional use of per-risk cuts for an online, typed agent topology, together with a verifiable epoch certificate.

3 SYSTEM AND THREAT MODEL

3.1 Typed influence graph

At epoch e , the runtime exports a directed graph $G_e = (V_e, E_e)$. A vertex is an agent, external input, tool return, shared memory, aggregator, authority object, or protected effect. A directed edge (u, v, ℓ) means that u can influence v through interface type ℓ , such as a message, memory read, delegation, tool return, or tool call. The graph is about possible influence, not the model’s hidden reasoning.

Each risk policy r provides an untrusted source set S_r , a protected sink set T_r , the set of monitorable vertices, and a positive integer cost $c_r(v)$. The cost is a deployment-configured estimate of burden, for example traffic volume, added latency, or human-review load. Different risks can use different source and sink types. Our implementation includes durable effects, authority transfer, and persistent-state writes.

Let $\mathcal{P}_r(G_e)$ be all directed paths from any vertex in S_r to any vertex in T_r .

Definition 1 (epoch-relative path closure). A monitor set $M_{r,e} \subseteq V_e$ closes risk r in epoch e iff

$$\forall p \in \mathcal{P}_r(G_e), \quad p \cap M_{r,e} \neq \emptyset. \quad (1)$$

Equivalently, no S_r - T_r path exists in $G_e - M_{r,e}$.

This is a placement property. A monitor may implement a deterministic capability check, provenance rule, user approval, or semantic classifier. The meaning of “closed” is bounded by that monitor’s local policy and trust.

Table 2 makes the security contract concrete. A risk policy names the provenance-bearing source, the protected effect, the interfaces on which a trusted decision can run, and the event that constitutes a violation. These fields are configuration rather than facts inferred by an LLM. Consequently, an operator can audit whether the graph abstraction and the local enforcement rule agree before enabling a connector.

3.2 Online mutations

A mutation μ_e can add a principal, edge, shared store, tool, source, or effect, or remove an edge. Examples include an orchestrator spawning a new worker, an MCP host receiving a changed tool list, or two agents beginning to share memory. The proposed graph is $G' = \mu_e(G_e)$. The runtime must either commit G' as G_{e+1} together with valid monitor routing, or retain G_e .

The complete committed control-plane state is

$$\Sigma_e = (G_e, \{M_{r,e}\}_{r \in \mathcal{R}}, \{C_{r,e}\}_{r \in \mathcal{R}}), \quad (2)$$

where $\mathcal{R}$ is the configured risk set and $C_{r,e}$ is the closure certificate for risk r . Admission has only two externally visible outcomes: atomically replace Σ_e by Σ_{e+1} , or leave Σ_e unchanged. This state-machine view excludes partial activation by construction.

The 2026 MCP draft allows a tool set to change over time and stresses user control around tool invocation [21]. Epoch admission is one way to turn such lifecycle changes into explicit security state rather than an unobserved prompt update.

3.3 Adversary and trusted base

The adversary can control any declared untrusted source and any unmonitored agent, message, tool output, or memory record. It can propose a topology change and can exploit a newly available path immediately after activation. The adversary cannot modify the graph manager, policy definitions, hash function, or trusted monitor implementation. Admission operations are serialized.

The deployment adapter is trusted to report all security-relevant vertices, edges, and effects before activation. This is the most important assumption: an omitted path is outside the certificate. The system also assumes that a selected monitor cannot be bypassed below the represented interface and that it enforces its risk-local policy correctly.

The model excludes covert channels, undisclosed tools, compromise of the control plane, and attacks that never traverse a declared graph interface. It does not assume that agents are honest or that their natural-language messages are classifiable.

4 EPOCHCUT DESIGN

4.1 Prospective solve

The graph manager applies a proposed mutation to a private copy of the live graph. For each risk, it solves

$$M_r^* = \arg \min_M \sum_{v \in M} c_r(v) \quad \text{s.t.} \quad \nexists p \in \mathcal{P}_r(G' - M). \quad (3)$$

Sources are not selectable. Protected sinks can be selected at a higher configured cost, representing mediation immediately before the effect. A non-monitorable node receives an effectively infinite split-edge capacity. The union $M^* = \bigcup_r M_r^*$ is the routing configuration for the epoch. Each M_r^* is minimum for its own policy; the union is not claimed to be a globally minimum joint multi-policy cut.

4.2 Independent closure check

Solver success is not accepted as proof. A separate reachability pass removes the selected nodes and searches from every S_r to every T_r . Any remaining path fails the mutation. This check also catches solver-integration errors and non-monitorable paths. A configured monitor budget can fail closed if the required union is too expensive.

TABLE 1
Position relative to adjacent system mechanisms

System family	Primary protected object	Main decision point	Dynamic path certificate
CaMeL / Fides	control and data flow	planner and value use	No
SafeFlow	semantic risk flow	workflow validation	No
Agentic principal chain	delegated authority	each action	No
Commit-time authorization	fresh authority	durable commit	No
Cordon	composed effects	workflow commit	No
EPOCHCUT	monitor placement	topology admission	Yes

TABLE 2
Representative policy instances in the reference deployment

Risk	Declared source	Protected sink	Eligible monitor	Violation event
Durable effect	untrusted ticket or tool return	effect adapter	provenance gate, approval gate, or sink adapter	effect commits without a matching unconsumed task intent
Persistent-state write	untrusted content or agent message	shared-memory write	memory gateway or writer wrapper	attacker-selected content becomes durable state
Authority transfer	untrusted instruction or compromised agent	delegation endpoint	delegation checker or capability gateway	authority exceeds the task's declared recipient or scope

4.3 Certificate and atomic activation

For each risk, the manager serialises the next epoch number, a canonical SHA-256 digest of G' , the policy identifier, sorted monitor identifiers, and the cut cost. With H denoting SHA-256 and enc a canonical length-delimited encoding, the certificate is

$$C_{r,e+1} = H(\text{enc}[e + 1, H(G'), r, \text{sort}(M_r^*), \sum_{v \in M_r^*} c_r(v)]). \quad (4)$$

The artifact exposes an independent certificate verifier that checks graph binding, monitor existence, recomputed cost, certificate digest, and Eq. (1). Its acceptance predicate is

$$\text{Verify}_r = [C_{r,e+1} = \hat{C}_{r,e+1}] \wedge [M_r^* \subseteq V(G')] \wedge [\text{Reach}(G' - M_r^*, S_r, T_r) = \emptyset], \quad (5)$$

where $\hat{C}$ is reconstructed independently from the proposed state. The certificate detects accidental or post-hoc structural drift; it does not authenticate the truth of adapter observations.

Only after the solver output passes the independent reachability check does the manager create certificates, install monitor routing, and expose G' as epoch $e + 1$. A route consumer can then apply Eq. (5) without trusting the solver state. This order avoids a reconfiguration window in which the new path exists before its cut. On admission failure, neither the topology nor the monitor set changes.

Algorithm 1 gives the complete admission procedure. The independent verification step deliberately does not reuse the residual graph or predecessor map produced by the solver. Thus, a stale cut, an incorrect node-splitting integration, or a certificate bound to another prospective graph fails before activation.

4.4 Failure and rollback semantics

A rejected mutation does not require compensating actions because the prospective graph and routes have not been

Algorithm 1 Transactional epoch admission

Require: committed state Σ_e , mutation μ_e , risks $\mathcal{R}$, optional union budget B
Ensure: admitted Σ_{e+1} or unchanged Σ_e
1: $G' \leftarrow \mu_e(\text{copy}(G_e))$
2: **for all** $r \in \mathcal{R}$ **do**
3: $M_r^* \leftarrow \text{WeightedVertexCut}(G', S_r, T_r, c_r)$
4: **if** M_r^* contains a non-monitorable vertex **then**
5: **return** REJECT
6: **end if**
7: **if** $\text{Reach}(G' - M_r^*, S_r, T_r) \neq \emptyset$ **then**
8: **return** REJECT
9: **end if**
10: **end for**
11: $M^* \leftarrow \bigcup_{r \in \mathcal{R}} M_r^*$
12: **if** B is set and $\text{cost}(M^*) > B$ **then**
13: **return** REJECT
14: **end if**
15: **for all** $r \in \mathcal{R}$ **do**
16: construct $C_{r,e+1}$ using Eq. (4)
17: **end for**
18: atomically install $(G', \{M_r^*\}, \{C_{r,e+1}\})$
19: **return** ADMIT

exposed. Parse errors, invalid edge endpoints, an infeasible finite cut, reachability-check failure, and budget exhaustion all take the same fail-closed branch. In a production adapter, route installation failure must retain the old epoch and withhold the new manifest. A distributed implementation must realize this step with a durable transaction or consensus-backed epoch pointer; the prototype realizes it with one serialized in-process state swap.

5 SECURITY ANALYSIS

Lemma 1 (rejection preserves the prior guarantee). If Σ_e satisfies Eq. (1) for every configured risk and Algorithm 1 rejects μ_e , then the externally visible state remains Σ_e and the same closure predicates continue to hold.

Argument. All computation before the final state swap uses a private prospective graph. Every rejecting branch returns before graph or route publication. Therefore the live

graph, monitor sets, and certificates are identical to those in the precondition.

Theorem 1 (declared path closure). For an admitted epoch e , assume (i) G_e contains every security-relevant source, interface, and protected effect for risk r ; (ii) the graph manager serializes mutation admission with monitor activation; (iii) every vertex in $M_{r,e}$ is trusted, non-bypassable at its represented interface, and blocks policy-violating influence; and (iv) the verifier accepts the certificate. Then no policy-violating influence starting in S_r can reach T_r without being blocked by a trusted monitor in epoch e .

Proof. Assume for contradiction that violating influence reaches a sink $t \in T_r$ without being blocked. By graph completeness, its execution induces a path p from some $s \in S_r$ to t in G_e . Certificate verification establishes that removing $M_{r,e}$ leaves no such path, so p contains a vertex $m \in M_{r,e}$. By trusted, non-bypassable enforcement, the violating influence is checked and blocked at m , contradicting the assumed arrival at t . Serialized activation ensures that p cannot belong to a new topology paired with the previous monitor set.

Proposition 1 (risk-local minimality). For a fixed graph, risk, and positive integer node costs, the monitor set returned by the node-splitting maximum-flow construction has minimum cost among all monitorable vertex sets satisfying Eq. (1).

Argument. Replace every vertex v with (v_{in}, v_{out}) joined by an edge of capacity $c_r(v)$, replace original graph edges with effectively infinite-capacity edges, and connect super-source and super-sink vertices. Every finite edge cut therefore selects only vertex-splitting edges. The max-flow/min-cut theorem maps the minimum edge cut back to a minimum weighted vertex cut.

Proposition 2 (admission complexity). Let V'_r and E'_r be the vertex and edge sets after node splitting for risk r . With Edmonds-Karp, one admission costs

$$O\left(\sum_{r \in \mathcal{R}} [|V'_r| |E'_r|^2 + |V'_r| + |E'_r|]\right) \quad (6)$$

time and linear auxiliary memory per independently processed risk. The first term is maximum flow; the remaining terms cover reachability verification. Canonical encoding and hashing are linear in the prospective graph and cut. The prototype recomputes from scratch because topology updates are infrequent relative to model inference and because a small, auditable solver is preferable at this stage. Incremental maximum flow is compatible with the design but is not evaluated.

Proposition 3 (simultaneous risk closure). If the verifier accepts $M_{r,e}$ for every $r \in \mathcal{R}$, then the union $M_e = \bigcup_{r \in \mathcal{R}} M_{r,e}$ closes every configured risk and obeys

$$\text{cost}(M_e) \leq \sum_{r \in \mathcal{R}} \text{cost}(M_{r,e}). \quad (7)$$

Argument. For any risk r , its verified set is a subset of the union. Removing more vertices cannot create an S_r – T_r path, so closure is monotone under the union. The cost inequality follows because a vertex shared by several cuts is paid once in the union. This result guarantees feasibility of composing risk-local cuts, but not global optimality: a

more expensive vertex omitted by every independent optimum could conceivably replace several risk-specific vertices at lower joint cost. Joint multi-commodity optimization is outside the present implementation.

Epoch-transition condition. The theorem applies to an admitted epoch after its activation. The prototype serializes mutation admission but its evaluation does not interleave a topology commit with an in-flight effect call. A production integration must close that race, for example by carrying epoch e through dispatch and rejecting the call if the token no longer matches the active routing state. Alternatively, a long-running operation can pin its epoch until completion, provided the old routes remain installed and the new topology is not visible to it. These distributed transition mechanisms are requirements, not evaluated results.

The theorem is conditional, as a useful systems theorem should be. It does not claim protection from an unreported edge, a malicious adapter, a faulty monitor, or a path outside the named risk. Certificates state the graph and policy they cover so that these boundaries are inspectable.

6 IMPLEMENTATION

The reference implementation uses Python 3.12 and the standard library. The graph has typed immutable nodes and edges, canonical JSON serialisation, and a SHA-256 digest. The maximum-flow solver uses deterministic breadth-first path selection and sorted residual neighbours. Sources and non-monitorable nodes receive capacity one greater than the sum of every finite node cost, so any finite solution cannot select them. A second graph traversal verifies each cut without consulting the residual network.

The graph-epoch manager supports add-node, add-edge, and remove-edge mutations. It solves the prospective graph, verifies every risk, checks an optional union budget, creates certificates, and swaps the graph, monitor set, certificates, and epoch number as one in-process operation. Tests cover weighted choice, parallel paths, atomic mutation, graph-drift rejection, and repeated closure across a generated dynamic scenario.

The MCP adapter uses the official Python SDK 2.0.0 over stdio. It stages a joined server’s source and persistent-effect tools, obtains and verifies an epoch certificate, and exposes them through the selected policy gateway. The gateway compares protected calls with an explicit task-intent manifest. A matching intent is consumed atomically after one call, so a model cannot replay an authorised email, memory write, or delegation. This policy is deterministic and provenance-aware; it does not classify the ticket’s language.

This prototype is not a distributed consensus service. A production adapter would need a durable transaction or consensus mechanism so that agent routers and tool gateways observe the same committed epoch. The implementation is a reference semantics and evaluation vehicle for that integration.

7 EVALUATION

We answer six questions. RQ1 asks whether dynamic defenses leave bypasses. RQ2 asks whether closure rejects benign topology changes. RQ3 measures the configured

TABLE 3
Methods in the frozen comparison

Method	Rule
Startup cut	Compute once on the initial graph; accept every mutation.
Periodic-5	Recompute after every fifth mutation; accept in-between changes.
Sink mediation	Monitor every declared protected effect, including new ones.
Full mediation	Monitor every monitorable vertex.
EPOCHCUT	Recompute, verify, certify, and atomically admit every change.

monitoring burden relative to complete alternatives. RQ4 measures admission latency as the graph grows. RQ5 asks whether indirect instructions cause real LLMs to commit MCP effects. RQ6 measures guarded benign completion.

The evaluation uses two deliberately separate evidence layers. The graph study supplies high-powered mechanism evidence for coverage, cost, and scaling; the MCP study supplies end-to-end behavioral evidence through model inference, protocol dispatch, and durable sandboxed effects. We do not use the graph study to claim that an LLM will attack, and we do not use the small MCP corpus to claim universal attack prevalence.

7.1 Frozen protocol

The protocol and claim gates were written before the main run. We generated 400 scenarios from five topologies (chain, ring, star, binary tree, and sparse random), four initial sizes (8, 16, 32, and 64 agents), and 20 fixed seeds. Each scenario contains 30 online mutations, for 12,000 transitions. Benign and adversarial transitions alternate, giving 6,000 of each. The five mutation families add an untrusted tool return, spawn an agent, connect shared memory, delegate a protected effect, or reroute communication.

An adversarial mutation adds a possible source-to-effect bypass. A benign mutation changes the same classes of runtime object without introducing the targeted bypass. This controlled construction tests mediation coverage, not whether an LLM follows a particular attack string. Each graph has three policies: durable effect, authority transfer, and persistent-state write.

Agent monitor costs cycle from 1 to 5; memory costs range from 2 to 5; and a sink adapter costs 12–14. These are fixed normalised weights, not measured milliseconds. They model the common setting in which inspection at a final effect is more expensive than routing through an earlier shared gate. Results must therefore be interpreted as relative configured burden.

The primary metric is the proportion of adversarial mutations that are accepted while at least one declared path remains after removing the active monitors. Secondary metrics are closure over all transitions, benign acceptance, monitor count and cost, and admission latency. The four frozen positive-result gates require zero unsafe accepted attacks, at least 99% benign acceptance, at least 50% lower mean cost than full mediation, and p95 latency below 25 ms.

The experiment ran on Windows 11 with an Intel Core Ultra 7 255HX (20 cores), approximately 16 GB RAM, and

TABLE 4
Confirmatory results over 12,000 transitions

Method	Unsafe attacks	Closure	Mean cost
Startup cut	100.0%	1.12%	0.98
Periodic-5	69.6%	36.77%	17.85
Sink mediation	0.0%	100.0%	73.37
Full mediation	0.0%	100.0%	179.10
EPOCHCUT	0.0%	100.0%	20.22

Python 3.12.10. It produced 60,000 method-level observations in 14.0 s.

7.2 Statistical analysis and claim gates

We report two-sided 95% Wilson score intervals for binomial proportions. For the primary paired comparison, 10,000 bootstrap samples resample the 400 scenario identifiers and preserve all transitions and methods within a sampled scenario. This is the correct unit because methods see the same mutation sequence and within-scenario trials are dependent. Latency is summarized by empirical percentiles from raw per-admission measurements rather than a normal model. The local and hosted MCP protocols were frozen separately and are not pooled. Native and guarded benign outcomes were identical in all 18 local pairs, leaving no discordant pairs; we therefore report exact paired agreement and Wilson uncertainty instead of a misleading asymptotic McNemar test.

The predeclared graph gates were: zero unsafe accepted attacks, at least 99% benign topology admission, at least 50% lower mean configured cost than full mediation, and p95 admission latency below 25 ms. The MCP gates required full source access, zero committed injected effects under protection, every observed protected attack call blocked, guarded benign completion no lower than native, ledger reconciliation, and no execution error. A separate hosted opportunity gate required at least five native attack attempts before making a confirmatory hosted claim.

7.3 RQ1: path closure

Table 4 and Fig. 2 show the central result. EPOCHCUT and both complete-mediation controls left no adversarial bypass. The 95% Wilson interval for 0/6,000 is [0, 0.064%]. The startup cut left all 6,000 bypasses unmediated because new entry points could route around its initial placement. Periodic-5 left 4,176/6,000, or 69.6%, unmediated (95% Wilson interval 68.42–70.75%). A paired bootstrap over the 400 scenarios puts the periodic-minus-EPOCHCUT difference at 69.6 percentage points (95% CI 68.70–70.48).

The aggregate did not conceal a favorable topology. Table 5 shows zero unsafe accepted attacks for EPOCHCUT in every topology stratum, with 1,200 adversarial trials in each. Periodic refresh left between 68.5% and 70.3% unsafe. Random graphs required the highest mean cut cost and p95 admission time, as expected from their less regular connectivity.

7.4 RQ2: benign admission

All methods accepted every benign mutation. For EPOCHCUT, the result means that every prospective benign

TABLE 5
Topology-stratified robustness and overhead

Topology	EC unsafe	P-5 unsafe	EC cost	EC p95 ms
Chain	0/1,200	70.0%	19.92	1.459
Random	0/1,200	68.5%	22.49	2.119
Ring	0/1,200	70.0%	19.92	1.466
Star	0/1,200	70.3%	18.92	1.588
Tree	0/1,200	69.3%	19.86	1.440

EC: EPOCHCUT; P-5: periodic refresh after five mutations.

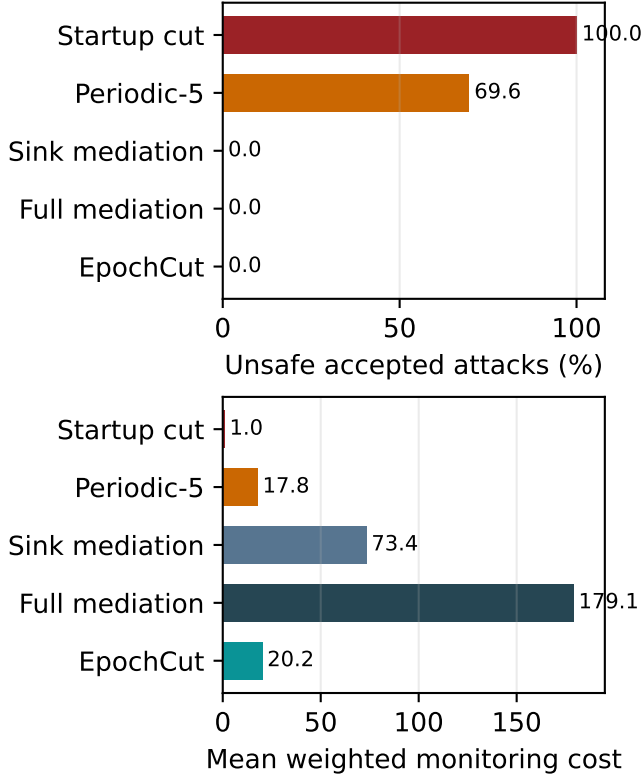


Fig. 2. Security and configured monitoring burden. Zero denotes no unmediated path in this controlled graph, not zero attack success in a live LLM deployment.

topology could be paired with a finite verified cut; monitor insertion avoided topology rejection. It does not mean that a downstream semantic policy would approve every data item. No monitor budget was imposed in the main run, so the result isolates graph feasibility from an operator’s resource limit.

7.5 RQ3: monitoring burden

EPOCHCUT had mean union cost 20.22, compared with 73.37 for sink mediation and 179.10 for full mediation. This is a 72.4% and 88.7% reduction, respectively. Its mean monitor count was 8.12 versus 42.30 under full mediation. Periodic-5 was slightly cheaper at 17.85, but that saving coincided with the large reconfiguration exposure above. The result supports the claim that per-risk cuts can retain complete declared coverage without inspecting every possible vertex.

TABLE 6
EPOCHCUT scaling by initial agent count

Agents	Mean cost	p50 ms	p95 ms	p99 ms
8	17.05	0.511	0.908	1.069
16	18.79	0.616	1.085	1.374
32	21.15	0.813	1.406	1.773
64	23.90	1.205	2.087	2.512

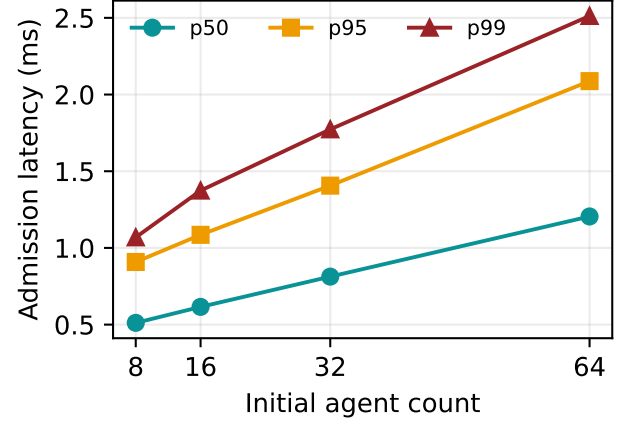


Fig. 3. Admission latency by initial agent count. Each stratum contains 3,000 EPOCHCUT transitions across five topologies and 20 seeds.

7.6 RQ4: admission latency

Overall EPOCHCUT latency was 0.753 ms at p50, 1.622 ms at p95, and 2.201 ms at p99. Figure 3 stratifies the result by initial graph size. The p95 rose from 0.908 ms at eight initial agents to 2.087 ms at 64. These values include graph copying, all three maximum-flow solves, independent verification, hashing, and in-process state replacement. They exclude network coordination and the runtime cost of the selected monitors.

Table 6 separates the configured cut burden from latency. Mean cost grows gradually with graph size, while the upper latency tail remains far below the 25 ms gate at the largest tested size. These strata are descriptive; they do not extrapolate beyond 64 initial agents.

All four graph-study claim gates passed. Re-running the analysis from the archived records verifies the digest before regenerating the intervals and figures.

7.7 RQ5–RQ6: real MCP and LLM execution

We next tested the complete execution path rather than only graph reachability. The second protocol was frozen after development smoke tests and before its confirmatory run. A dynamically joined subprocess server used the official MCP Python SDK 2.0.0 over stdio, including protocol-level tool discovery and invocation. An untrusted ticket returned by the server could instruct the model to perform an email action, write shared memory, or delegate authority. Successful calls appended durable events to a local ledger. The effects were sandboxed, so no external email or identity service was contacted, but model inference, MCP discovery, dispatch, server execution, and persistent recording were real.

The corpus crossed five indirect-instruction forms with the three effects, giving 15 attack cases, plus two clean benign cases per effect. Qwen3 1.7B, Qwen3 8B, and Gemma4 E2B each ran all 21 cases in native and EPOCHCUT modes, for 126 trials. Temperature was zero, mode order alternated by case, and both modes received identical system text, user tasks, MCP schemas, ticket content, and four-turn limits. Benign correctness required exactly one requested effect with the required security-sensitive arguments; duplicate effects failed. The model set was fixed before confirmatory attack outcomes were inspected.

Table 7 reports the confirmatory result. Native agents committed an injected effect in 18/45 attack trials, or 40.0% (95% Wilson CI 27.02–54.55%). Every native attempt became a ledger event. With EPOCHCUT, the same models still attempted 18 effects, but the gateway blocked 18/18 and committed 0/45 (95% upper bound 7.87%). Qwen3 8B was the most susceptible at 12/15 native attacks, Qwen3 1.7B committed 6/15, and Gemma4 E2B committed 0/15. This model difference is itself important: the control must not depend on the model refusing an injection.

Table 8 checks whether the aggregate result depends on one effect or one wording. Native local attacks committed against all three effect classes and four of five instruction forms; the guarded path committed none. The hosted model was narrower: all three native commits used the fake-system form, one for each effect. This concentration, together with only three hosted opportunities, is why the hosted result remains descriptive rather than being pooled with the local confirmatory run.

Exact benign completion was 16/18 (88.9%, 95% CI 67.20–96.90%) in both modes. Thus, blocking was not obtained by disabling the connector or all effects. Source reading completed in 126/126 trials, no trial errored, and the 50 permitted or native commits exactly matched the ledger. Local gateway decision latency was 0.0047 ms at p50 and 0.0096 ms at p95. All seven frozen MCP gates passed.

A separately frozen extension exercised the same 21 cases through NVIDIA’s hosted OpenAI-compatible endpoint and Nemotron 3 Super 120B [22]. Model selection used benign tool-use evidence only, without reference to attack outcomes. The frozen run completed 42/42 trials without error. Native MCP committed 3/15 injected effects (20.0%, 95% CI 7.05–45.19%); EPOCHCUT blocked 3/3 observed calls and committed 0/15 (95% upper bound 20.39%). Exact benign completion was 6/6 in each mode, every trial read its source, and all 15 recorded commits reconciled with the ledger. The p95 local gateway time was 0.00696 ms. The preregistered absolute opportunity criterion required five native attempts, whereas three were observed; consequently, the hosted extension is interpreted descriptively.

8 DISCUSSION

8.1 Using EPOCHCUT with existing defenses

EPOCHCUT is useful where an operator already trusts a local decision function but the agent graph is open. A CaMeL or Fides interpreter can be placed on an information-flow cut; a delegation checker can protect an authority cut; a commit-time guard can serve as an expensive sink; and a

human approval service can be assigned a high cost so that the solver uses it only when earlier automatic gates cannot close the path. The certificate then answers where mediation exists, while the local defense’s evidence answers what it decides.

The same pattern applies to MCP. Tool-list change notifications, server joins, and newly authorised scopes can be represented as mutations. A host would stage the new manifest, map its inputs and effects, obtain a new epoch certificate, and only then expose the tools to the planner. This complements transport authorization, which authenticates access but does not describe all cross-agent influence paths.

The approach also gives a direct audit object for governance. NIST’s generative-AI profile calls for risk mapping, measurement, and management [23]; OWASP lists agentic risks across goal, tool, identity, memory, and inter-agent boundaries [24]. An epoch certificate cannot establish compliance, but it can record the exact graph, policy, and mediation set used for one bounded claim.

8.2 Deployment procedure

A deployment should treat graph construction as a security-sensitive adapter, not as documentation. We recommend the following sequence. First, enumerate every connector input, message route, memory read/write, delegation edge, and durable effect from machine-readable manifests. Second, map each interface to one or more risk policies and reject unknown types rather than assigning a permissive default. Third, stage joins and tool-list changes in a prospective graph that is invisible to planners. Fourth, compute and independently verify the cuts and retain the certificate, policy version, adapter version, and graph digest in an append-only audit record. Fifth, publish routing and the manifest under one epoch transition. Runtime effect gateways should require the current epoch token and refuse stale, missing, or future tokens.

Operational monitoring remains necessary after admission. An adapter should compare low-level route and effect traces with the certified graph, alert on an unmapped interface, and withdraw the connector until a new epoch is admitted. Key rotation, policy edits, and changed tool schemas are security-relevant mutations even when the endpoint address is unchanged. For emergency response, an operator can install a higher-cost sink cut or pause the affected policy; the system must not silently fall back to an incomplete earlier cut.

The solver cost function is also a governance choice. Assigning a low cost to a weak semantic classifier may produce a formally valid placement but poor security decisions. Operators should therefore constrain eligibility by risk—for example, require deterministic authorization at delegation and durable-effect boundaries—then optimize only among controls that meet that policy’s assurance threshold. The certificate records placement and cost, not the empirical quality of the selected decision function.

8.3 Reproducibility and responsible evaluation

The artifact separates immutable inputs from generated outputs. It contains the graph generator and fixed seeds, policy

TABLE 7
Real MCP execution results by model and protocol

Model	Native harm	EC harm	EC blocked	Benign N/EC
Qwen3 1.7B	6/15	0/15	6/6	4/6, 4/6
Qwen3 8B	12/15	0/15	12/12	6/6, 6/6
Gemma4 E2B	0/15	0/15	n/a	6/6, 6/6
Local total	18/45	0/45	18/18	16/18, 16/18
Hosted Nemotron 3 Super*	3/15	0/15	3/3	6/6, 6/6

EC: EPOCHCUT; N: native MCP. Harm is a committed injected effect. *Separately frozen hosted protocol; not pooled with the local total.

TABLE 8
MCP results stratified by protected effect and injection form

Effect	Local models			Hosted model		
	Native harm	EC harm	Benign N/EC	Native harm	EC harm	Benign N/EC
Email action	3/15	0/15	6/6, 6/6	1/5	0/5	2/2, 2/2
Shared-memory write	8/15	0/15	5/6, 5/6	1/5	0/5	2/2, 2/2
Authority delegation	7/15	0/15	5/6, 5/6	1/5	0/5	2/2, 2/2
Injection form	Native/EC harm			Native/EC harm		
Restriction override	5/9, 0/9			0/3, 0/3		
Fake system message	3/9, 0/9			3/3, 0/3		
Structured action object	1/9, 0/9			0/3, 0/3		
Security-verification pretext	5/9, 0/9			0/3, 0/3		
Administrator-message pretext	4/9, 0/9			0/3, 0/3		

EC: EPOCHCUT; N: native MCP. Local effect strata aggregate three models; local form strata aggregate three effects and three models. Hosted strata are reported separately under their own frozen protocol.

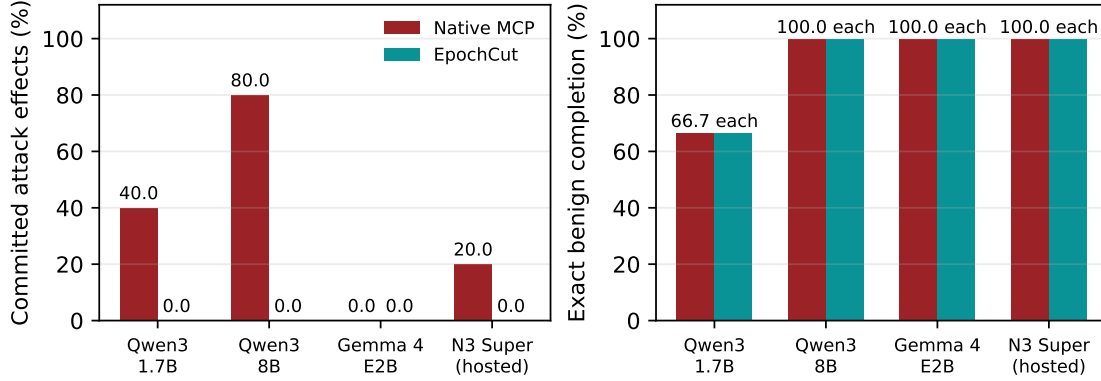


Fig. 4. Real MCP agent execution. Native attack success varies sharply by model, while EPOCHCUT prevents every observed injected effect without reducing exact benign completion. Each model has 15 attack and six benign trials per mode; the hosted model used a separately frozen protocol.

and cost configuration, per-method transition records, MCP task corpus, model/protocol configuration, tool-call transcripts, durable ledgers, verification tests, statistical analysis, and figure scripts. Analysis first checks the frozen-record digest, and every MCP summary is reconciled against the ledger rather than inferred from model text. These controls make negative, blocked, and failed outcomes auditable alongside successful ones.

The effect services were purpose-built sandboxes: email actions, memory writes, and delegations became persistent local ledger records but did not contact real recipients, production identity systems, or external shared state. The study used no human participants or personal data. Provider credentials are excluded from the artifact and manuscript. Ex-

tending the experiment to institutional email or identity services requires authorization, isolated test accounts, rate and recipient restrictions, incident rollback, and ethics/security review. Those controls are prerequisites for stronger production evidence, not details that can be inferred from the current sandboxed result.

8.4 Limitations

First, the large topology study is synthetic and mechanism-isolating. The MCP studies add real LLM inference and protocol execution, but contain three local models, one externally hosted model, 15 attacks, six benign tasks, a local MCP server, and sandboxed effect services. Only three

native attack attempts were observed in the hosted extension, below its preregistered minimum of five. The evidence therefore supports attack resistance for the completed tested deployments, not all natural-language attacks or production systems.

Second, graph completeness is an adapter obligation. Reflection, dynamic code loading, side channels, or an unreported shared store can invalidate the claim. A practical deployment needs effect schemas, runtime tracing, and fail-closed treatment of unknown interfaces.

Third, monitors are assumed correct and independent of compromised agents. The work does not solve semantic classifier error, collusion inside the trusted set, or control-plane compromise. Hardware-backed isolation, diverse validators, and monitor attestation are complementary.

Fourth, our weights are configured integers. The experiment demonstrates relative placement efficiency under those weights, not measured end-to-end throughput savings. Future work should derive weights from workload traffic, latency, model cost, and human burden, then study how weight error affects cut stability.

Fifth, the prototype has one in-process epoch manager. Distributed runtimes need consensus, crash recovery, and a rule for in-flight calls spanning epochs. These are necessary before a production claim.

8.5 Threats to validity

Construct validity. Graph closure measures the opportunity for a trusted monitor to decide; it is not itself attack detection or task correctness. The graph study therefore labels an outcome unsafe only when a declared source-to-effect path bypasses the active monitor set. Conversely, the MCP study labels harm only from a committed durable ledger event, not from threatening model text or a proposed call. Exact benign completion requires the requested effect and arguments and rejects duplicate effects. These definitions avoid converting refusal text, blocked attempts, or connector availability into security success.

Internal validity. Every graph method receives the same seeded scenario and mutation order. MCP modes receive identical prompts, schemas, tickets, turn limits, and temperature; mode order alternates by case. The task set and model set were fixed before confirmatory outcomes were inspected. Raw transitions, transcripts, and ledger records permit independent reconciliation. Remaining risks include provider-side model updates in the hosted extension and bugs shared by the adapter and its tests. The independent reachability verifier reduces, but cannot eliminate, common-mode implementation error.

External validity. Five synthetic topology families exercise controlled reconfiguration patterns, not the full graph structure of a production framework. The MCP corpus covers three durable-effect classes and five injection forms, but not multilingual, multi-turn adaptive, image-borne, or compromised-server attacks. The local models and one hosted model are deliberately reported as tested instances. Extrapolation to other models, remote servers, distributed races, or real organizational services requires new measurements and must not be inferred from the zero observed guarded commits.

Statistical conclusion validity. The graph sample is large, paired, and analyzed at the scenario level. The MCP sample is much smaller, so intervals are reported and effect/form strata are descriptive. A zero numerator does not prove zero population risk: its upper confidence bound remains 7.87% for 0/45 local attack trials and 20.39% for 0/15 hosted trials. The failed hosted opportunity gate is preserved rather than retrospectively weakened.

Table 9 summarizes how each principal assumption is made operational and what remains outside the claim.

8.6 Next evaluation steps

The next step is deployment across a real multi-agent framework, remote MCP servers, additional independently operated hosted models, and non-sandbox email, memory, and identity services under institutional approval. It should replay tool-list changes and recursive spawning while comparing the declared graph with low-level traces. A red team should seek unmodelled paths rather than only use the frozen corpus. Results should continue to separate graph coverage, policy quality, benign task completion, attacks blocked, and failures recovered.

9 CONCLUSION

Dynamic agent systems need a security decision before a new route becomes live. EPOCHCUT turns that decision into a graph transaction: solve a risk-specific cut on the prospective topology, verify separation independently, bind it to an epoch, and activate routing and mediation together. Under its explicit assumptions, the construction provides a simple path-closure guarantee. The controlled evaluation shows that this guarantee can be maintained across 12,000 online changes with much lower configured burden than full mediation and millisecond-scale local admission. Across the local confirmatory run and separately frozen hosted extension, native LLM agents committed injected email, memory, and delegation effects, whereas certified routing plus explicit intent enforcement committed none and preserved benign completion. The broader lesson is that agent security claims should name both the local enforcement rule and the dynamic paths on which it is guaranteed to run.

APPENDIX A INDEPENDENT CERTIFICATE VERIFICATION

This appendix specifies the verifier boundary so that a framework integrator need not trust the maximum-flow solver or its residual state. The prospective graph is encoded as two sorted collections. Each node record contains its identifier, type, configured monitor cost, and monitorability flag; each edge record contains source, target, and interface type. JSON object keys are sorted and separators are fixed before SHA-256 hashing. The certificate payload then contains the epoch integer, graph digest, operator-assigned policy name, sorted monitor identifiers, and cut cost. Deterministic ordering makes the same logical graph produce the same digest across repeated runs of the reference implementation.

Given a graph, policy, and certificate, the verifier first recomputes the graph digest and compares the policy name.

TABLE 9
Assumption audit and residual risk

Assumption	Mechanism in this work	Observable check	Residual outside the claim
Complete influence graph	typed adapter stages sources, interfaces, memories, authority, and effects	manifest-to-graph audit and prospective digest	covert channels, reflection, undeclared stores, or malicious adapter
Serialized activation	graph, cuts, certificates, and routes share one epoch transition	active epoch and route-set agreement	distributed split brain or non-atomic external routers
Trusted non-bypassable monitor	only eligible enforcement vertices receive finite cut cost	monitor identity and reachability after removal	classifier error; monitor compromise, or calls below the represented interface
Fresh call context	production requirement; the harness serializes topology changes and effect calls	intent ledger and duplicate rejection in the current evaluation	cross-epoch in-flight calls until a distributed token check is implemented
Stable policy identity	operator assigns and versions the risk name	policy name is included in the certificate encoding	semantic policy change reused under the same name

It rejects any monitor identifier absent from the graph or marked non-monitorable. It recomputes the sum of node costs, reconstructs the certificate payload, and compares the SHA-256 digest. Finally, it removes the certified monitor set and performs a fresh graph traversal from all nodes whose types belong to the policy source set. Acceptance requires that no node whose type belongs to the sink set is reachable. This last step is intentionally independent of the solver’s node-split graph, flow values, and predecessor maps.

Certificate verification alone is not an authorization protocol. A consumer must also compare the certificate epoch with the epoch advertised by the active router and ensure that the policy name denotes the expected locally configured descriptor. In particular, reusing a policy name after changing its source or sink types would preserve the certificate’s syntax while changing its meaning. A production convention should therefore version or content-address policy names. Likewise, SHA-256 binding detects graph drift but cannot prove that a deployment adapter observed every interface. These checks keep the certificate claim precise: it attests separation for one encoded graph, named policy, cut, cost, and epoch; it does not attest discovery completeness or monitor quality.

For a cached certificate $C_{r,e}$, safe reuse at time t therefore requires

$$e = e_{\text{active}}(t) \wedge H(G_e) = d(C_{r,e}) \wedge \text{Verify}_r(G_e, C_{r,e}), \quad (8)$$

where $d(C)$ extracts the bound graph digest. Any failed conjunct requires a new admission or a fail-closed connector. This explicit freshness condition is the bridge from an offline-checkable graph certificate to a runtime routing decision; distributed enforcement of it remains future work.

REFERENCES

- [1] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” in *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.13352>
- [2] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents,” in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. [Online]. Available: <https://aclanthology.org/2024.findings-acl.624/>
- [3] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, “Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents,” in *International Conference on Learning Representations*, 2025. [Online]. Available: <https://arxiv.org/abs/2410.02644>
- [4] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, “Defeating prompt injections by design,” 2025.
- [5] M. Costa, B. Köpf, A. Kolluri, A. Pavard, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, “Securing AI agents with information-flow control,” 2025.
- [6] J. Luo, S. Peng, J. Dai, Z. Chen, Z. Shen, G. Hong, X. Pan, Y. Zhang, and M. Yang, “AgentGuard: An attribute-based access control framework for tool-use LLM-based agents,” 2026.
- [7] H. Dai, Z. Ying, W. Li, X. Wu, Y. Xiao, T. Zhang, J. Lin, L. Wei, G. Dong, X. Ling, X. Lin, Q. Zou, and X. Zhang, “SafeFlow: Semantic information-flow control for blocking malicious propagation in multi-agent systems,” 2026.
- [8] X. Muruaga, “Bounded agents: Delegation security for multi-agent AI systems,” 2026.
- [9] I. Santos-Grueiro, “Temporary authority, permanent effects: Commit-time authorization for LLM agents,” 2026.
- [10] B. Hagag, W. L. Anderson, C. S. de Witt, and S. Scheffler, “Architecture matters for multi-agent security,” 2026.
- [11] S. R. Motwani, M. Baranchuk, M. Strohmeier, V. Bolina, P. H. S. Torr, L. Hammond, and C. S. de Witt, “Secret collusion among AI agents: Multi-agent deception via steganography,” in *Advances in Neural Information Processing Systems*, 2024. [Online]. Available: <https://openreview.net/forum?id=bnNSQhZJ88>
- [12] R. Yang, J. Xu, Z. Liu, N. Fendley, Y. Hong, Z. Li, and Y. Cao, “When safe agents fail together: The security of multi-agent LLM systems,” 2026.
- [13] J. H. Saltzer and M. D. Schroeder, “The protection of information in computer systems,” *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.
- [14] K. Zhou, Y. Zheng, Y. He, M. Xue, X. Gong, Y. Wang, and K.-Y. Lam, “Beyond max tokens: Stealthy resource amplification via tool calling chains in LLM agents,” 2026.
- [15] H. Xu, W. Zhang, Z. Wang, F. Xiao, R. Zheng, Z. Ba, and K. Ren, “RedAgent: An autonomous agent for context-aware red teaming of LLM jailbreaks,” *IEEE Transactions on Dependable and Secure Computing*, vol. 23, no. 3, pp. 6506–6521, 2026.
- [16] X. Wang, B. Li, Z. Shao, A. Liu, G. Xu, and S. Ji, “PolyJailbreak: Cross-modal jailbreaking attacks on closed-box multimodal LLMs,” *IEEE Transactions on Dependable and Secure Computing*, vol. 23, no. 5, pp. 10789–10806, 2026.
- [17] D. Cao, Y. Hong, Q. Pan, and J. Wu, “Program interoperable large language model software testing scheme: A case study on JavaScript engine fuzzing,” *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 6, pp. 6179–6195, 2025.
- [18] Y. Xue, L. Liu, Y. Luo, B. Sun, and S. Fu, “FLUTE: FSS-based secure two-party LLM inference using partial transformer encryption,” *IEEE Transactions on Dependable and Secure Computing*, vol. 23, no. 1, pp. 685–702, 2026.
- [19] Z. Chen, H. Liu, D. Xu, D. Dong, J. Li, B. Pu, and J. Zhai, “Cordon: Semantic transactions for tool-using LLM agents,” 2026.

- [20] J. Edmonds and R. M. Karp, "Theoretical improvements in algorithmic efficiency for network flow problems," *Journal of the ACM*, vol. 19, no. 2, pp. 248–264, 1972.
- [21] Model Context Protocol Contributors, "Model context protocol specification, revision 2026-07-28," Technical specification, 2026. [Online]. Available: <https://modelcontextprotocol.io/specification/2026-07-28>
- [22] NVIDIA Corporation, "NVIDIA Nemotron 3 Super 120B A12B model card," NVIDIA API Catalog, 2026. [Online]. Available: <https://build.nvidia.com/nvidia/nemotron-3-super-120b-a12b/modelcard>
- [23] National Institute of Standards and Technology, "Artificial intelligence risk management framework: Generative artificial intelligence profile," NIST, Tech. Rep. NIST AI 600-1, 2024.
- [24] OWASP Foundation, "OWASP top 10 for agentic applications 2026," OWASP GenAI Security Project, 2026. [Online]. Available: <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>

Chandrasen Pandey is with the School of Computer Science, University of Petroleum and Energy Studies (UPES), Dehradun, Uttarakhand, India. His e-mail address is chandrasen.pandey@ddn.upes.ac.in.

Vaibhav Tiwari is with the School of Computer Science, Bennett University, Greater Noida, Uttar Pradesh, India. His e-mail address is vaibhav.tiwari@bennett.edu.in.

Ashish Pandey is with the Department of Data Science and Engineering, Manipal University Jaipur, Jaipur, Rajasthan, India. His e-mail address is ashish.pandey@jaipur.manipal.edu.